Extra-Credit Assignment No. 1

Title: ElGamal Signature
Course: Hardware Security
Instructor: Dr. Siamak Mohammadi
Teaching Assistant: Alireza Ramezani
Academic Term: Second semester, academic year 1405–1406 (as stated in the original)

English translation of "تمرین امتیازی شماره 1.pdf"

Introduction

The security of cryptographic mechanisms in hardware systems, especially Hardware Security Modules (HSMs) and Secure Elements, depends not only on the theoretical strength of the underlying cryptographic algorithms, but also to a great extent on how they are implemented in practice.

The history of practical attacks shows that many security failures occur not because of a theoretical weakness in the algorithms, but because of implementation errors, improper management of randomness, and violations of security assumptions.

One important example is the **ElGamal digital signature**, whose security depends directly on the unpredictability of the random **nonce** (`k`) used for each signature. In hardware scenarios, this value is usually generated inside the device and never leaves it. However, if the same nonce is used to sign two different messages—for example, because of a weak random-number generator (RNG), a design flaw, or an incorrect optimization—the private signing key may be recovered computationally and deterministically, without solving the discrete logarithm problem.

This assignment is designed to develop a deeper understanding of implementation vulnerabilities in cryptographic signatures used in secure hardware. Students will examine a deliberately flawed implementation of hashed ElGamal signatures, provided as `buggy_elgamal.py`. It simulates an HSM or Secure Element: the private key is never directly disclosed, but the device mistakenly reuses a nonce when signing different hash values.

How the ElGamal Signature Works

The ElGamal digital signature scheme is a public-key signature algorithm based on the multiplicative group modulo a prime number and the assumed difficulty of the **discrete logarithm problem (DLP)**. In practice, ElGamal signatures are applied to a message hash rather than to the message itself, improving efficiency and providing a fixed-size input.

1. System Parameters and Keys

Let:

- `p` be a large prime number.
- `g` be a generator of the multiplicative group `Z_p*`.

- `x ∈ [1, p−2]` be the private signing key.
- `y = g^x mod p` be the public verification key.

The public parameters are **`(p, g, y)`**, while the private key `x` is kept secret within the signing device (such as an HSM or Secure Element).

2. Signing Procedure

To sign a message `m`, the signer first computes its hash:

`h = hash(m)`, where `h ∈ [0, p−2]`.

Step 1 — Choose a random nonce: Select a fresh random value

`k ∈ [1, p−2]` such that `gcd(k, p−1) = 1`.

The nonce must be **unique and unpredictable for every signature**.

Step 2 — Compute the first signature component:

`r = g^k mod p`

Step 3 — Compute the second signature component:

`s = k^(-1) × (h − x×r) mod (p−1)`

The ElGamal signature is the pair **`(r, s)`**.

3. Signature Verification

To verify `(r, s)` on hash `h`, the verifier checks whether

`g^h ≡ y^r × r^s (mod p)`.

- If the equality holds, the signature is valid.
- Otherwise, the signature is rejected.

4. Correctness of the Scheme

Since `y = g^x` and `r = g^k`, we have:

`y^r × r^s = g^(x×r) × g^(k×s) = g^(x×r + k×s)`.

By the definition of `s`:

`s ≡ k^(-1)(h − x×r) (mod p−1)`

`⇒ k×s ≡ h − x×r (mod p−1)`

Therefore:

`x×r + k×s ≡ h (mod p−1)`

and hence:

`y^r × r^s ≡ g^h (mod p)`.

This proves that honestly generated signatures always verify correctly.

5. The Role of the Nonce and Its Security Implications

ElGamal signature security relies on two important assumptions:

1. The difficulty of the discrete logarithm problem.
2. The freshness and secrecy of the nonce `k`.

If the same nonce is used to sign two different hashes, `h1` and `h2`, both signatures have the same value `r = g^k mod p`. This produces two linear equations involving the same unknown values, `k` and `x`. An attacker can then recover the nonce and compute the private key efficiently.

Consequently, nonce reuse can completely compromise ElGamal signature security, even when the private key never leaves the secure hardware.

Assignment Questions

Question 1 — Manual Proof (20 points)

Prove by hand that the secret key and the nonce can be recovered.

Important information:

- The public parameters are `(p, g, y)`; `x` is the private signing key.
- The device (HSM or Secure Element) signs hash values in the range `[0, p−2]`.
- In `buggy_elgamal.py`, `keygen()` returns the public parameters while `x` remains inside the device.
- The function `sign_hash(h, p, g) -> (r, s)` signs a hash value.
- The function `verify_hash(h, (r, s), p, g, y) -> bool` determines whether the signature on `h` is valid.

Question 2 — Recover the Nonce and Private Key in Python (30 points)

Using Python, recover the values of **`k`** and **`x`**.

Guidance: You may import and use `modinv`, `sign_hash`, `keygen`, and `verify_hash` from `buggy_elgamal.py` in your code to recover `x` and `k`.

In your report, explain each part of the code you wrote and include screenshots of the outputs obtained.

Question 3 — Forge an ElGamal Signature (30 points)

Use the recovered value of **`x`** to forge an ElGamal signature.

Explain the different parts of your code and provide a reasoned explanation of why the forgery is possible.

Conceptual Questions (20 points total)

Question 1. Why does reusing a random nonce compromise the security of ElGamal signatures?

Question 2. What is the difference between forging a valid signature and merely computing a counterfeit signature pair `(r, s)`? Why should the forgery target `(r1, s1)` or `(r2, s2)`?

Question 3. Why does ElGamal compute `s` modulo `p−1` rather than modulo `p`?

Question 4. What fundamental assumption guarantees the security of ElGamal signatures?

Required Submissions

1. A handwritten/manual solution to Question 1.
2. A Python source file named `attacker_elgamal.py`, together with an explanation of how the code works.
3. A PDF report answering all assignment questions, covering both theoretical and practical parts.

Translation note: This is an English rendering of the original Persian assignment. Mathematical expressions and technical requirements have been retained. The signer described above is deliberately vulnerable for educational purposes.